1. (2 Punkte) Liste ist unser selbstgebastelter Datentyp. Wir nummerieren die Einträge der Liste mit Zeiger `anf` an der Stelle 0.

   a. Welche Zahlenfolge findet sich in den Listenelementen?
   b. Gib den Index des Elements an, auf das der Zeiger `pos` zeigt.

```
a = Liste()
a.insert(1)
a.insert(2)
a.advance()
a.insert(3)
a.insert(4)
a.reset()
a.advance()
a.insert(6)
```

2. (2 Punkte) Liste ist unser selbstgebastelter Datentyp. Wir nummerieren die Einträge der Liste mit Zeiger `anf` an der Stelle 0.

   a. Welche Zahlenfolge findet sich in den Listenelementen?
   b. Gib den Index des Elements an, auf das der Zeiger `pos` zeigt.

```
a = Liste()
a.insert(3)
a.advance()
a.insert(2)
a.insert(5)
a.insert(3)
a.reset()
a.advance()
a.delete()
a.insert(4)
a.advance()
```

3. (3 Punkte) Liste ist unser selbstgebastelter Datentyp.

   a. Welche Zahlenfolge findet sich in den Listenelementen?
   b. Welchen Inhalt hat das aktuelle Element?

```
a = Liste()
a.insert(4)
a.insert(3)
a.advance()
a.insert(6)
a.reset()
a.advance()
a.delete()
a.insert(9)
a.advance()
a.advance()
a.insert(7)
a.reset()
a.advance()
a.advance()
```

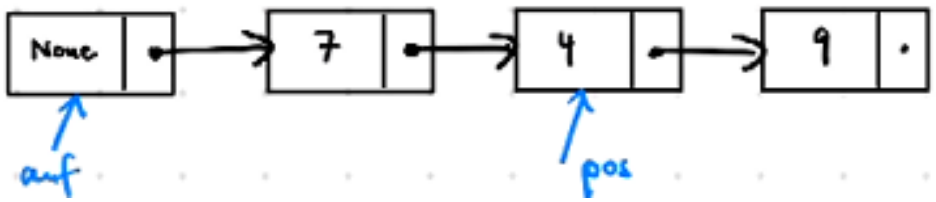
4. (3 Punkte) Liste ist unser selbstgebastelter Datentyp.

```
a = Liste()
a.insert(5)
a.insert(8)
a.advance()
a.insert(2)
a.advance()
a.delete()
```

   (a) (2 Punkte) Zeichne die Einträge der Liste a mit den Zeigern `anf` und `pos` nach diesen Anweisungen.

   (b) (1 Punkt) Was liefert jetzt `a.endpos()`?

5. (5 Punkte) Liste ist unser selbstgebastelter Datentyp. Das Bild zeigt die Situation der Liste a.



a. Was liefern die folgenden Anweisungen:

```
print(a.elem())
print(a.endpos())
```

b. Zeichne die Situation nach der Ausführung von a.insert(12)

c. Was liefern jetzt die folgenden Anweisungen:

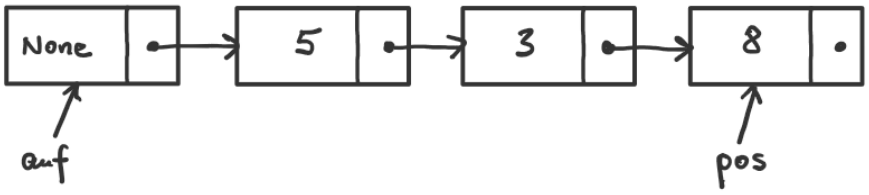
```
print(a.elem())
print(a.endpos())
```

d. Wir gehen nochmal von der Anfangssituation aus. Zeichne die Situation nach Ausführung von a.delete()

e. Was liefern jetzt die folgenden Anweisungen:

```
print(a.endpos())
print(a.elem())
```

6. (4 Punkte) Liste ist unser selbstgebastelter Datentyp. Das Bild zeigt die Situation der Liste a.



a. Was liefern die folgenden Anweisungen:

```
print(a.endpos())
print(a.elem())
```

b. Es werden folgende Anweisungen ausgeführt:::

```
a.reset()
a.insert(9)
```

Zeichne die entstandene Situation.

c. Was liefert jetzt die Anweisung a.elem()???

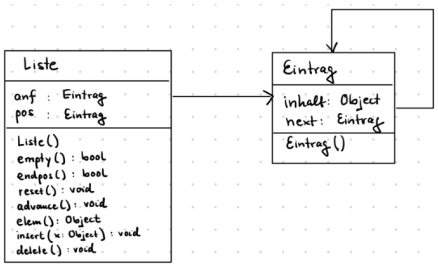
7. (3 Punkte) Die Liste a enthält die Zahlen 3 7 5. Die Schleife soll alle Elemente ausgeben.

```
a.reset()
while not a.endpos():
    a.advance()
    print(a.elem())
```

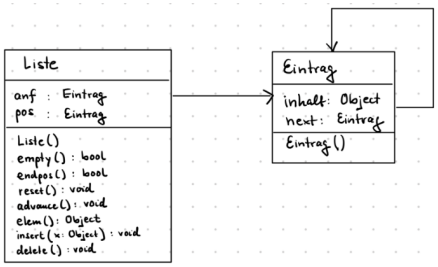
(a) (2 Punkte) Was geschieht stattdessen?

(b) (1 Punkt) Korrigiere die Schleife.

8. (4 Punkte) Das Bild zeigt das UML-Diagramm unseres Datentyps Liste. Implementiere in Python die Klassen Eintrag und die Klasse Liste (nur Konstruktor und Methode delete). Achte dabei auf mögliche Fehlersituationen.



9. (4 Punkte) Das Bild zeigt das UML-Diagramm unseres Datentyps Liste. Implementiere in Python die Klassen Eintrag und die Klasse Liste (nur Konstruktor und die Methoden endpos und advance). Achte dabei auf mögliche Fehlersituationen.



10. (4 Punkte) Die Klasse Eintrag und der Konstruktor unserer Klasse Liste sind gegeben.

```
class Eintrag:
    def __init__(self):
        self.inhalt = None
        self.next = None

class Liste:
    def __init__(self):
        self.anf = Eintrag()
        self.pos = self.anf
```

Implementiere die Methoden `elem` und `insert`. Achte dabei auf mögliche Fehlersituationen.

11. (3 Punkte) In der Methode `insert` sind zwei Zeilen vertauscht.

```
def insert(self, x):
    hilf = Eintrag()
    hilf.inhalt = x
    self.pos.next = hilf
    hilf.next = self.pos.next
```

(a) (1 Punkt) Welche Zeilen müssen getauscht werden?

(b) (2 Punkte) Was passiert mit dieser falschen Methode, wenn die folgenden Anweisungen ausgeführt werden? Begründe.

```
a = Liste()
a.insert(1)
a.insert(2)
while not a.endpos():
    print(a.elem())
    a.advance()
```

12. (6 Punkte) Liste ist unser selbstgebastelter Datentyp. Benutze nur die Methoden der Liste.
    (a) (2 Punkte) Erzeuge eine Liste a mit den Zahlen von 1 bis 100 in aufsteigender Reihenfolge.
    (b) (2 Punkte) Berechne die Summe der Elemente von a.
    (c) (2 Punkte) Verschiebe alle durch 3 teilbaren Zahlen aus a in eine neue Liste b. Die Reihenfolge der Zahlen soll erhalten bleiben. Danach ist die Summe der Elemente von a gleich 3367 und die von b gleich 1683.

13. (3 Punkte) Die Funktion `anzahl(a, x)` bekommt eine Liste a (unser Datentyp) und gibt zurück, wie oft x in der Liste vorkommt. Benutze nur die Methoden der Liste. Implementiere die Funktion.

14. (3 Punkte) Die Funktion `ersetze(a, alt, neu)` ersetzt in der Liste a (unser Datentyp) jedes Vorkommen von alt durch neu. Die Reihenfolge der Elemente bleibt erhalten. Benutze nur die Methoden der Liste. Implementiere die Funktion.

15. (4 Punkte) Die Funktion `doppelte_entfernen(a)` löscht aus der sortierten Liste a (unser Datentyp) alle doppelten Elemente, sodass jeder Wert nur noch einmal vorkommt. Aus 1 1 2 3 3 3 5 wird 1 2 3 5. Die Liste kann leer sein. Benutze nur die Methoden der Liste. Implementiere die Funktion.

16. (4 Punkte) Die Klasse `FeatureListe` erbt von Liste. Sie hat zusätzlich ein Attribut `gerade`, das genau dann `True` ist, wenn die Anzahl der gespeicherten Listenelemente gerade ist. Schreibe die Klasse `FeatureListe`.

17. (4 Punkte) Die Klasse `FeatureListe` erbt von Liste und hat die zusätzliche Methode `laenge`, die die Anzahl der gespeicherten Elemente zurückgibt. Das aktuelle Element soll sich dabei nicht ändern. Schreibe die Klasse `FeatureListe`.

18. (3 Punkte) a. Gib für die beteiligten Verweisboxen die Attributwerte nach Ablauf der untenstehenden Anweisungen an. (Angabe in der Form: inhalt = 'e', oben = a, unten = b)
b. Welche Ausgabe erscheint auf der Konsole?

```
class VerweisBox:
    def __init__(self, inhalt, unten=None, oben=None):
        self.inhalt = inhalt
        self.unten = unten
        self.oben = oben
        if unten is not None: self.unten.oben = self
        if oben  is not None: self.oben.unten = self

    def __str__(self):
        return self.inhalt

a = VerweisBox('a')
b = VerweisBox('b')
c = VerweisBox('c',a,b)
d = VerweisBox('d',b,c)
print(a.unten, b.unten.oben, b.oben.oben.oben, c.unten, c.oben.oben.unten.unten.unten)
```

19. (3 Punkte) Welche Ausgabe erscheint auf der Konsole?

```
class VerweisBox:
    def __init__(self, inhalt, unten=None, oben=None):
        self.inhalt = inhalt
        self.unten = unten
        self.oben = oben
        if unten is not None: self.unten.oben = self
        if oben  is not None: self.oben.unten = self

    def __str__(self):
        return self.inhalt

a = VerweisBox('a')
b = VerweisBox('b')
c = VerweisBox('c',a,b)
d = VerweisBox('d',c,a)
print(c.unten.oben)
print(b.unten.oben.unten.unten)
print(d.unten.unten.unten)
```

20. (3 Punkte) Eine Python-Liste ist ein Array. Unsere Liste besteht aus verketteten Einträgen. Entscheide und begründe kurz.
    (a) (1 Punkt) Ein Programm greift sehr oft über die Position i auf ein Element zu. Welche Datenstruktur passt besser?
    (b) (1 Punkt) Ein Programm geht durch eine lange Warteliste und fügt dabei oft Elemente ein oder entfernt sie. Welche Datenstruktur passt besser?
    (c) (1 Punkt) Warum lohnt sich die binäre Suche auf unserer Liste nicht, auch wenn sie sortiert ist?

21. (4 Punkte) Erkläre in ein bis zwei Sätzen.
    (a) (1 Punkt) Was ist ein Abstrakter Datentyp?
    (b) (1 Punkt) Warum zeigt `pos` auf den Eintrag vor dem aktuellen Element und nicht auf das aktuelle Element selbst?
    (c) (1 Punkt) Wozu dient der Dummy-Eintrag am Anfang der Liste?
    (d) (1 Punkt) Warum nennt man die Liste eine rekursive Datenstruktur?